

Python Fundamentals: The Survival Guide

Weeks 2–4: Logic, Functions, & Files

CMPT 211

Concordia University of Edmonton

Friday, Feb 6, 2026

The 3 Pillars of Programming

Forget the fancy terms. Every program is built from just three blocks:

1. Logic

(Week 2)



Making Decisions

(if / else)

2. Structure

(Week 3)



Grouping Tasks

(def function)

3. Safety

(Week 4)



Handling Errors

(try / except)

1. Logic (The if Block)

The Rule: Python uses **Indentation** (Space) to group code.

```
temperature = 30

# The colon (:) starts the block
if temperature > 25:
    # 4 SPACES HERE -> Inside the 'if'
    print("It is hot.")
    print("Drink water.")

# NO SPACES HERE -> Outside the 'if'
print("Check complete.")
```

If you forget the spaces, Python will crash (IndentationError).

2. Loops (The for Block)

The Rule: "For every item in this list, do this action."

```
# A list of numbers
prices = [10, 5, 20]

total = 0

# 'p' is a variable we create right here
for p in prices:
    # Inside the loop (Indented)
    print(f"Adding: {p}")
    total = total + p

print(f"Final Total: {total}")
```

3. Functions (The def Block)

Concept: A Function is a named tool.

- **Input:** What the tool needs (Parameters).
- **Output:** What the tool gives back (Return).

```
# DEFINITION (Building the tool)
def check_password(input_text):
    if input_text == "secret123":
        return True
    else:
        return False

# EXECUTION (Using the tool)
is_allowed = check_password("12345") # Returns False
```

The Most Common Mistake

Print vs. Return

Print puts text on the screen for the human.

Return sends data back to the computer code.

Wrong (Dead End)

```
def add(a, b):  
    print(a + b)  
  
x = add(2, 2)  
# x is Empty (None)!
```

Correct (Chainable)

```
def add(a, b):  
    return a + b  
  
x = add(2, 2)  
# x is 4
```

4. Safety (The try Block)

Programs crash when data is bad. We use try to catch the crash.

```
user_input = "hello"

try:
    # 1. Try the dangerous code
    number = int(user_input)
    print("Success!")

except ValueError:
    # 2. Run this if step 1 fails
    print("Error: That is not a number.")

print("I am still running!")
```

5. Files (The with open Block)

How to save data so it lasts after the program stops.

```
message = "Log Entry: System Started"

# "w" = Write Mode (Create new file)
# "a" = Append Mode (Add to end)

try:
    with open("log.txt", "w") as f:
        f.write(message)
        print("Saved successfully.")

except Exception:
    print("Could not save file.")
```


Putting It All Together

A "Simple Calculator" using all 3 weeks of topics.

```
def save_result(result):  
    with open("results.txt", "a") as f:  
        f.write(f"Result: {result}\n")  
  
def divide(a, b):  
    try:  
        answer = a / b  
        return answer  
    except ZeroDivisionError:  
        return "Error: Cannot divide by zero"  
  
# Main Program  
val = divide(10, 2)  
if val != "Error":
```

The Safety Net: Try & Except

Why do we need this?

- Normally, if an error happens, the program **crashes** (stops immediately).
- We want the program to **survive** errors (like a user typing text instead of numbers).

The Logic Flow:

- **try:** "Attempt to run this dangerous code."
- **...If it works:** Skip the 'except' block and continue.
- **...If it fails:** Stop immediately and jump to the 'except' block.
- **except:** "Run this emergency plan so we don't crash."

What is a JSON file?

Context: We don't want to hard-code settings (like passwords) inside our Python script. We save them in a separate text file called **JSON**.

The Anatomy:

- { }: Curly Braces act as the container.
- "Key":: The name of the setting. *Must use double quotes!*
- **Value**: The actual data (Number, String, True/False).

Example (config.json)

```
1 {  
2     "app_name": "  
        StudentPortal",  
3     "version": 1.5,  
4     "dark_mode": true  
5 }
```

How Python sees it:

Python reads this file and turns it into a **Dictionary**.
`settings["app_name"]` gives you `"StudentPortal"`.

The Goal for Today

We are combining our 3 survival skills (Logic, Functions, Safety) to solve two real-world problems:

① **Dynamic Settings (DevOps):**

How to change your program's rules *without* rewriting code.

Tool: JSON Reader

② **Robust Data Loading (Data Engineering):**

How to read a messy file without crashing when errors appear.

Tool: Safe CSV Reader

The Problem: Hardcoding

Bad Practice: Writing settings directly inside the Python code.

```
def check_grade(score):  
    # What if the university changes the pass mark?  
    # You have to edit the Python code and re-save it.  
    if score >= 50:  
        return "Pass"
```

The Solution: Move the "50" to a separate text file called config.json.

Step 1: Anatomy of a JSON File

Create a text file named `config.json`. It is just a text file, but it follows strict rules.

The Syntax Rules:

- **Curly Braces** { }: These act as the "container." Everything must be inside them.
- **Double Quotes** " ": Every **Key** (the name) MUST use double quotes. Single quotes (') will cause a crash.
- **Colon** :: Separates the Name from the Data.
- **Comma** ,: Means "There is another item coming."
Note: The last item does NOT have a comma!

```
{  
    "pass_mark": 50,  
    "max_students": 30,  
    "semester": "Winter  
                2026"  
}
```

Step 1 (Continued): Understanding the Data

Why did we choose these specific fields?

1 "pass_mark": 50

- **Type:** Integer (Number).
- **Purpose:** We will use this in mathematical logic (e.g., `if score > 50`).

2 "semester": "Winter 2026"

- **Type:** String (Text).
- **Purpose:** Information to display on the screen. It needs quotes because it is text.

3 "features": ["dark_mode", "beta"]

- **Type:** List (Array).
- **Purpose:** JSON can hold lists too, just like Python!

Step 2: The Python Function

We need a tool (Function) to fetch these settings.

```
import json    # 1. Import the tool library

def load_settings(filename):
    """Reads a JSON file and returns a Dictionary."""

    # 2. Open the file in 'Read' mode (r)
    with open(filename, "r") as f:

        # 3. Convert Text File -> Python Dictionary
        data = json.load(f)

    # 4. Send the dictionary back
    return data
```


Step 3: Using It (Dynamic Logic)

Now our code adjusts automatically!

```
# 1. Load the settings
my_config = load_settings("config.json")

# 2. Extract the specific threshold
threshold = my_config["pass_mark"]

# 3. Use the variable, NOT the number 50
print(f"The pass mark is: {threshold}")

score = 60
if score >= threshold:
    print("You passed!")
```

The Problem: Messy Data

Real data files are never perfect.

Example File (grades.csv):

- 85 (Good)
- 90 (Good)
- N/A (**CRASH!** Not a number)
- 45 (Good)

We need a function that reads the good lines and **skips** the bad ones without stopping.

The Strategy: The Double Shield

We need two layers of `try/except` protection.

- ❶ **Outer Shield:** Handles if the *file* is missing.
- ❷ **Inner Shield:** Handles if a *line* is corrupt.

Step 1: The Outer Shield (File Safety)

First, ensure we can even open the file.

```
def read_grades(filename):  
    print(f"Reading {filename}...")  
  
    try:  
        # Try to open the file  
        with open(filename, "r") as f:  
            # (Loop will go here later...)  
            pass  
  
    except FileNotFoundError:  
        # If file is missing, do this  
        print("Error: File does not exist!")
```

Step 2: The Inner Shield (Data Safety)

Now, inside the loop, we protect the data conversion.

```
# Inside the 'with open' block:
for line in f:

    # CLEAN: Remove whitespace (\n)
    text = line.strip()

    try:
        # DANGER: Convert text to Integer
        score = int(text)
        print(f"Read score: {score}")

    except ValueError:
        # If conversion fails, run this:
        print(f"Skipping bad line: {text}")
```

The Complete Robust Function

```
def robust_reader(filename):  
    try:  
        with open(filename, "r") as f:  
            for line in f:  
                try:  
                    # Inner Shield: Bad Data  
                    val = int(line.strip())  
                    print(f"Loaded: {val}")  
                except ValueError:  
                    print("Skipping corrupt data.")  
  
    except FileNotFoundError:  
        # Outer Shield: Missing File  
        print("Critical Error: File missing.")
```

Checklist for the Lab

To complete the next tasks, remember:

- **Import JSON:** You need `import json` at the top.
- **Read Mode:** Use `"r"` inside `open()`.
- **Indentation:** The `try` block pushes code to the right. The inner `try` pushes it even further right.
- **Don't Panic:** If you see `"ValueError"`, it just means your Inner Shield is working!

The Final Goal: ETL

We don't just want to read data. We want to ****save**** it.

The Concept: ETL (Data Engineering)

- 1 **Extract:** Read raw data from CSV (The Grades).
- 2 **Transform:** Filter out the bad lines (The Inner Shield).
- 3 **Load:** Save the clean data into JSON (The Config File).

[Image of ETL pipeline diagram]

Step 3: Saving Data (The JSON Writer)

We need a function to write the grades **into** the settings file.

```
def save_results(filename, current_settings, new_data):  
    """  
    Saves the list of grades into the config dictionary.  
    """  
  
    # 1. THE MAGIC LINE (Assigning the Key)  
    # We create a new folder called "class_results"  
    current_settings["class_results"] = new_data  
  
    # 2. Save back to the file  
    with open(filename, "w") as f:  
        # indent=4 makes it look pretty  
        json.dump(current_settings, f, indent=4)
```

Understanding the "Magic Line"

Dictionary Assignment

```
current_settings["class_results"] = new_data
```

This single line does three things:

- **Finds** the dictionary in memory.
- **Creates** a new key named "class_results" (if it doesn't exist).
- **Fills** it with the list of grades we extracted from the CSV.

The Result: Before & After

Before (Input)

```
{  
    "pass_mark": 50  
}
```

After (Output)

```
1 {  
2     "pass_mark": 50,  
3     "class_results": [  
4         85,  
5         90,  
6         45  
7     ]  
8 }
```

We successfully merged our Code, our CSV data, and our JSON settings!

The Translation Guide: JSON ↔ Python

When you run `json.load()`, Python automatically translates the data types. You must know these equivalents to avoid bugs.

JSON (The File)	Python (The Code)	Critical Note
<code>{ ... }</code>	<code>dict</code>	Dictionary
<code>[...]</code>	<code>list</code>	List
<code>"text"</code>	<code>str</code>	Must use Double Quotes ("")
<code>10 / 5.5</code>	<code>int / float</code>	Numbers are the same
<code>true / false</code>	<code>True / False</code>	Capitalization changes!
<code>null</code>	<code>None</code>	null becomes None

The "Frozen" vs. "Thawed" Concept

Think of JSON as **Dehydrated Food**.

- You can't eat it (use it) while it is dehydrated.
- You must add water (parse it) to turn it into real food (variables).

1. The JSON String (Frozen)

```
import json

# Just a string of text!
raw_data = '{"id": 101, "active": true}'

print(type(raw_data))
# Output: <class 'str'>
```

2. The Python Dict (Thawed)

```
1 # Parse the string
2 real_data = json.loads(
3     raw_data)
4
5 print(type(real_data))
6 # Output: <class 'dict'>
7
8 # NOW we can use it
9 print(real_data["id"])
```

The characters { and : are just letters in the string until Python reads them!